Informe de Laboratorio 08

Tema: Supabase + CRUD + API REST + Fetch API

Nota

Integrantes	Escuela	Asignatura
Michael Alexander Vasquez Villarreal Franklin Ronald Ulloa Chura Kevin Aldair Solis Florez Gabriel Alberto Coaguila Quicaña	Escuela Profesional de Ingeniería de Sistemas	Introducción al Desarrollo Web Semestre: II Código: 2501210

Laboratorio	Tema	Duración
08	Supabase + CRUD + API REST + Fetch API	2 semanas (10 horas)

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	21 de octubre de 2026	04 de noviembre de 2026

1. Tarea

Implementar en el avance del proyecto **UNSA Conecta** un módulo CRUD Full Stack integrado con Supabase, enfocado representativamente en la entidad **Tutores**, que cumpla los siguientes objetivos:

- Modelar e implementar la base de datos relacional utilizando la plataforma Supabase.
- Conectar la aplicación Node.js/Express con Supabase utilizando su cliente oficial (`@supabase/supabase-js`), delegando la seguridad y prevención de Inyección SQL al ORM.
- Consumir los cinco endpoints de la API RESTful desde la interfaz web mediante la Fetch API (`async/await`), actualizando el DOM sin recargar la página y verificando los resultados directamente en el panel de Supabase.

2. Equipos, Materiales y Temas Utilizados

- Editor de código: Visual Studio Code.
- Entorno de ejecución: Node.js.
- Framework backend: Express.js.
- Base de datos: Supabase (BaaS).
- Librerías: `@supabase/supabase-js`, `dotenv` y `express`.
- Herramienta de BD: Supabase Dashboard (SQL Editor, Table Editor).
- Control de versiones: Git y GitHub.
- Frontend: HTML5, JavaScript nativo (Fetch API) y Tailwind CSS.

Temas de la guía: Tema 13 (Bases de Datos Relacionales y Node.js), Tema 14 (Arquitectura Web y CRUD) y Tema 15 (APIs REST con Node.js, Express y Fetch API).

3. URL de Repositorio GitHub

- **Repositorio completo:** https://github.com/MaicolVasquez/uc
- **Laboratorio 07:** https://github.com/MaicolVasquez/uc/blob/main/informes/IDWeb_lab07_avance_proyecto.pdf
- **Laboratorio 08:** https://github.com/MaicolVasquez/uc/blob/main/informes/IDWeb_lab08_avance_proyecto.pdf

4. Trabajo Colaborativo

El laboratorio se desarrolló en un único repositorio de GitHub. Siguiendo las observaciones del docente, **no se realizaron commits directos en la rama `main`**, salvo en el archivo `README.md` principal. Cada integrante trabajó en su propia rama y los cambios se integraron a `main` mediante *Pull Requests*. Luego, cada rama se actualizó con la última versión de `main` para evitar conflictos.

Integrante	Rol	Tareas en este laboratorio
Michael Alexander Vasquez Villarreal	Backend	Cliente Supabase, variables de entorno y configuración general.
Gabriel Alberto Coaguila Quicaña	Backend	Script SQL en Supabase, controlador CRUD y validación en Dashboard.
Franklin Ronald Ulloa Chura	Frontend	Interfaz HTML y renderizado de la tabla dinámica.
Kevin Aldair Solis Florez	Frontend	Lógica con Fetch API y refresco del DOM.

Tabla 1: Roles y tareas del equipo.

Código 1: Flujo de trabajo con Git seguido por cada integrante

```
# 1) Partir siempre de la ultima version de main
git checkout main
git pull origin main

# 2) Trabajar en la rama propia
git checkout feature/Michael_Vasquez
git add .
git commit -m "feat: CRUD integrado con el cliente oficial de Supabase"
git push origin feature/Michael_Vasquez

# 3) Abrir un Pull Request en GitHub (rama -> main) y aprobarlo

# 4) Actualizar la rama propia con los cambios que otros integraron en main
git checkout feature/Michael_Vasquez
git pull origin main
```

Explicación:

- `git pull origin main`: trae a la copia local los últimos cambios de `main`, para no trabajar sobre una versión desactualizada.
- `git commit` y `git push` se hacen siempre sobre la rama del integrante, nunca sobre `main`.
- Pull Request: es la forma de integrar el trabajo a `main` de manera revisada y paulatina. El profesor clona únicamente la rama `main`, por lo que esta debe contener el proyecto completo y funcional.
- Último paso: se vuelve a traer `main` a cada rama, de modo que todas las ramas queden actualizadas.

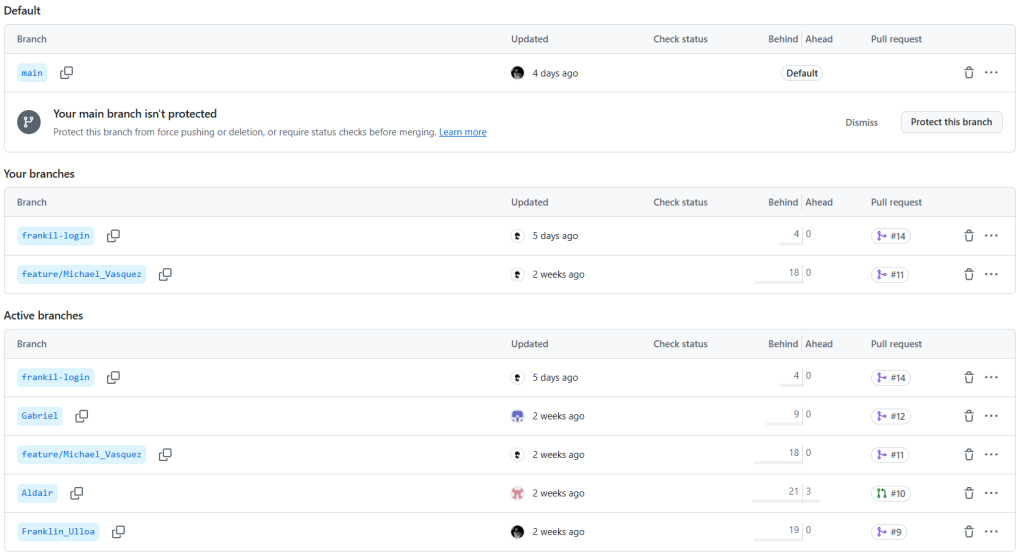


Figura 1: Historial de ramas y commits en GitHub.

5. Marco Conceptual

Bases de datos relacionales y Supabase. Una base de datos relacional organiza la información en tablas formadas por registros y atributos. En nuestro caso, usamos **Supabase**, un Backend-as-a-Service (BaaS) que envuelve una base de datos PostgreSQL. Permite definir restricciones avanzadas y expone una API REST segura para interactuar con los datos.

Cliente ORM y Abstracción de Conexiones. En lugar de mantener manualmente un Pool de conexiones TCP y escribir consultas SQL crudas, se utiliza el cliente oficial `@supabase/supabase-js`. Este cliente se comunica mediante solicitudes HTTP a la API interna de Supabase, la cual gestiona por detrás su propio pool de conexiones y el enrutamiento hacia la base de datos.

Inyección SQL y ORMs. La inyección SQL (SQLi) es neutralizada automáticamente al utilizar el cliente de Supabase. Al no concatenar cadenas SQL directas en el backend, sino pasar objetos JavaScript a métodos como `.insert()` o `.eq()`, la API de Supabase se encarga de parametrizar y sanear los datos antes de que toquen el motor de PostgreSQL.

API REST y CRUD. Una API REST expone recursos mediante URLs y utiliza los métodos HTTP para indicar la operación. La Tabla 2 muestra la correspondencia entre las operaciones CRUD y los métodos de Supabase.

Operación	Método HTTP	Endpoint	Método Supabase
Listar	GET	`/api/tutores`	`.select()`
Obtener uno	GET	`/api/tutores/:id`	`.select().eq()`
Crear	POST	`/api/tutores`	`.insert()`
Actualizar	PUT	`/api/tutores/:id`	`.update().eq()`
Eliminar	DELETE	`/api/tutores/:id`	`.delete().eq()`

Tabla 2: Correspondencia CRUD, HTTP y métodos de Supabase.

Fetch API. Es la interfaz del navegador (`fetch()`) para realizar solicitudes HTTP asíncronas

basadas en promesas. Combinada con `async/await` permite consultar la API y actualizar el DOM sin recargar la página completa.

6. Desarrollo del Laboratorio: Implementación SQL en UNSA Conecta

6.1. Estructura del proyecto

El módulo CRUD se integró al proyecto existente delegando el acceso a datos al cliente de Supabase. Los archivos marcados como **NUEVO** o **MODIFICADO** reflejan el alcance del laboratorio 08.

Código 2: Estructura de directorios del proyecto

```
uc/
|--- backend/
|    |--- controllers/
|    |    +--- tutoresController.js    (NUEVO: logica CRUD con Supabase)
|    |--- data/
|    |    +--- userStore.js
|    |--- middleware/
|    |    +--- auth.js
|    |--- routes/
|    |    |--- api.js
|    |    |--- auth.js
|    |    +--- tutores.js              (NUEVO: rutas REST)
|    |--- .env                         (credenciales; SE ENVIA al repositorio)
|    |--- .env.example                 (plantilla; MODIFICADO)
|    |--- .gitignore                   (MODIFICADO)
|    |--- db.js                        (NUEVO/MODIFICADO: Cliente Supabase)
|    |--- package.json                 (MODIFICADO)
|    +--- server.js                    (MODIFICADO: monta las rutas)
|--- bd/
|    |--- bd.sql                       (NUEVO: script de Supabase)
|    +--- Readme.md
|--- fronend/
|    |--- tutorias.html                (MODIFICADO: interfaz CRUD)
|    +--- pub/js/
|         |--- tutoring.js             (MODIFICADO: logica con Fetch API)
|         |--- data.js
|         +--- layout.js
+--- informes/
```

Archivo	Responsabilidad
`bd/bd.sql`	Script SQL que crea las tablas (incluyendo tutores) y tipos ENUM en Supabase.
`backend/db.js`	Inicializa el cliente de Supabase leyendo las credenciales del `.env` y exporta la instancia.
`backend/controllers/tutoresController.js`	Contiene los cinco manejadores CRUD; cada uno ejecuta métodos del ORM de Supabase.
`backend/routes/tutores.js`	Asocia cada método HTTP y URL con su manejador del controlador.
`backend/server.js`	Configura Express y monta las rutas bajo el prefijo `/api`.
`fronend/tutorias.html`	Estructura de la página: formulario de alta/edición y grid/tabla.
`fronend/pub/js/tutoring.js`	Consume la API con `fetch` y actualiza el DOM de manera asíncrona.

Tabla 3: Archivos principales del CRUD y su responsabilidad.

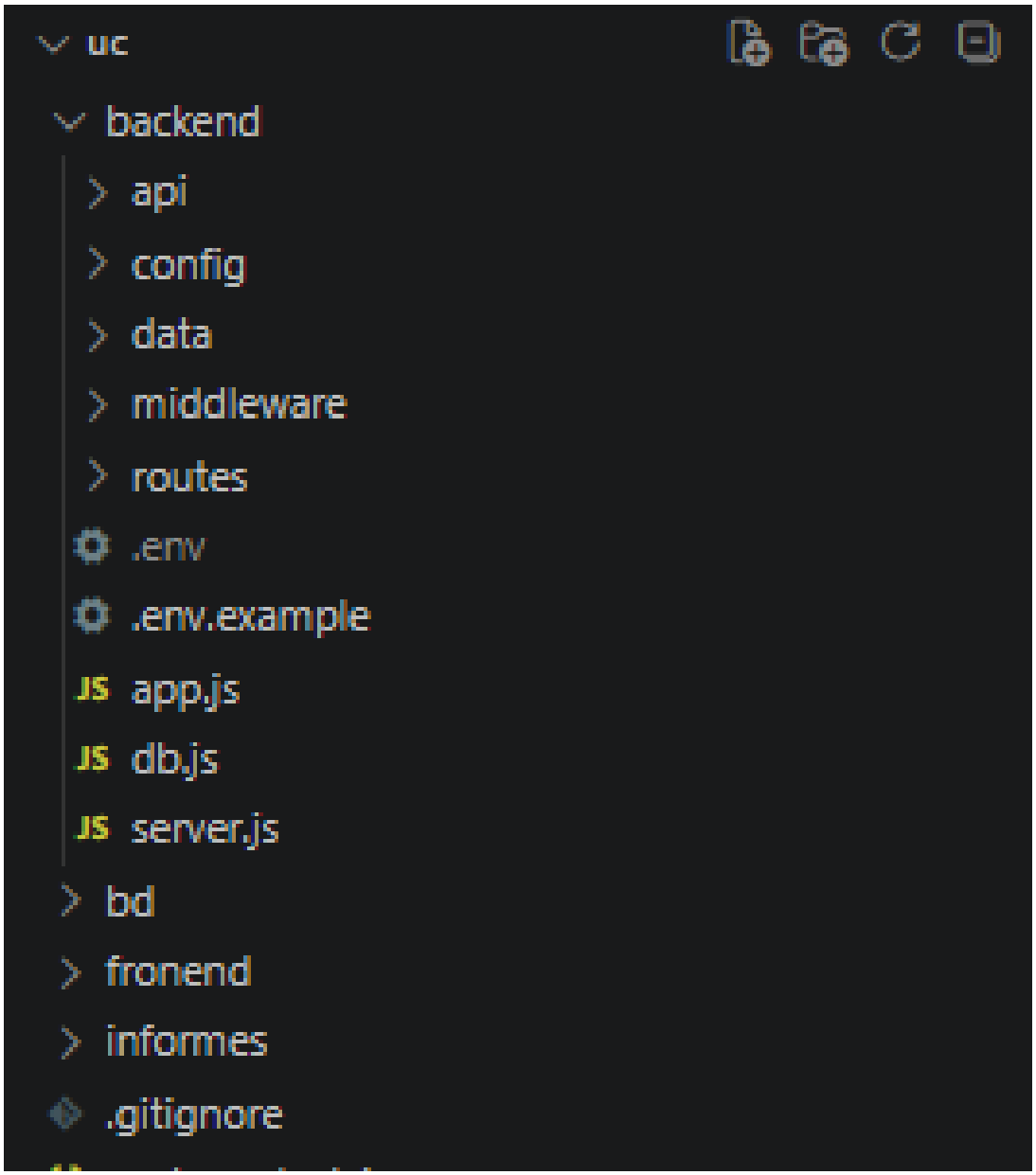


Figura 2: Estructura del proyecto en VS Code.

6.2. Diseño de la base de datos PostgreSQL en Supabase

Para garantizar un entorno real, la base de datos fue gestionada utilizando Supabase. A través del SQL Editor se ejecutó el siguiente script relacional, que abarca el uso de tipos ENUM, UUID y foráneas:

Código 3: bd/bd.sql

```
-- ========================================
-- TIPOS ENUM
-- ========================================
```

```sql
CREATE TYPE rol_usuario AS ENUM ('estudiante', 'admin');
CREATE TYPE estado_aprobacion AS ENUM ('pendiente', 'aprobado', 'rechazado');

-- ========================================
-- PERFILES
-- ========================================
CREATE TABLE perfiles (
    id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    rol        rol_usuario NOT NULL DEFAULT 'estudiante',
    nombre     VARCHAR NOT NULL,
    creado_en  TIMESTAMP NOT NULL DEFAULT now()
);

-- ========================================
-- TUTORES (Entidad base para nuestro CRUD)
-- ========================================
CREATE TABLE tutores (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    usuario_id      UUID NOT NULL REFERENCES perfiles(id) ON DELETE CASCADE,
    nombre_completo VARCHAR NOT NULL,
    carrera         VARCHAR,
    semestre        VARCHAR,
    campus          VARCHAR,
    cursos          VARCHAR,
    modalidad       VARCHAR,
    precio          VARCHAR,
    telefono        VARCHAR,
    bio             VARCHAR,
    estado          estado_aprobacion NOT NULL DEFAULT 'pendiente'
);

-- (Se incluyen de manera similar las tablas punto_hueco, objetos_perdidos, asesorias...)

-- ========================================
-- ÍNDICES (para optimizar búsquedas)
-- ========================================
CREATE INDEX idx_tutores_usuario      ON tutores(usuario_id);
CREATE INDEX idx_tutores_estado       ON tutores(estado);
```

Explicación del código:

- `CREATE TYPE ... AS ENUM`: Limita los valores posibles para roles y estados, delegando la integridad de datos a la BD.
- `id UUID PRIMARY KEY DEFAULT gen_random_uuid()`: Reemplaza al clásico `SERIAL` garantizando IDs globalmente únicos e impidiendo ataques de enumeración.
- `nombre_completo VARCHAR NOT NULL`: texto obligatorio que no puede ser insertado como NULL.
- `REFERENCES perfiles(id) ON DELETE CASCADE`: Garantiza la integridad referencial. Si un perfil se borra, sus registros asociados como tutor se borran automáticamente.
- `CREATE INDEX`: Mejora drásticamente el rendimiento cuando las consultas REST filtran por `estado` (ej: listar solo los aprobados).

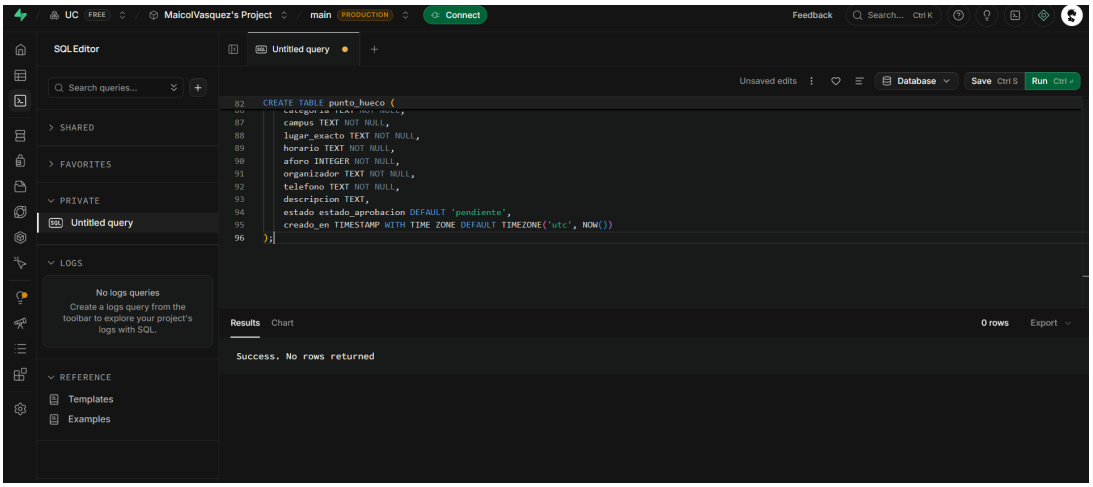


Figura 3: Script ejecutado en el SQL Editor de Supabase.

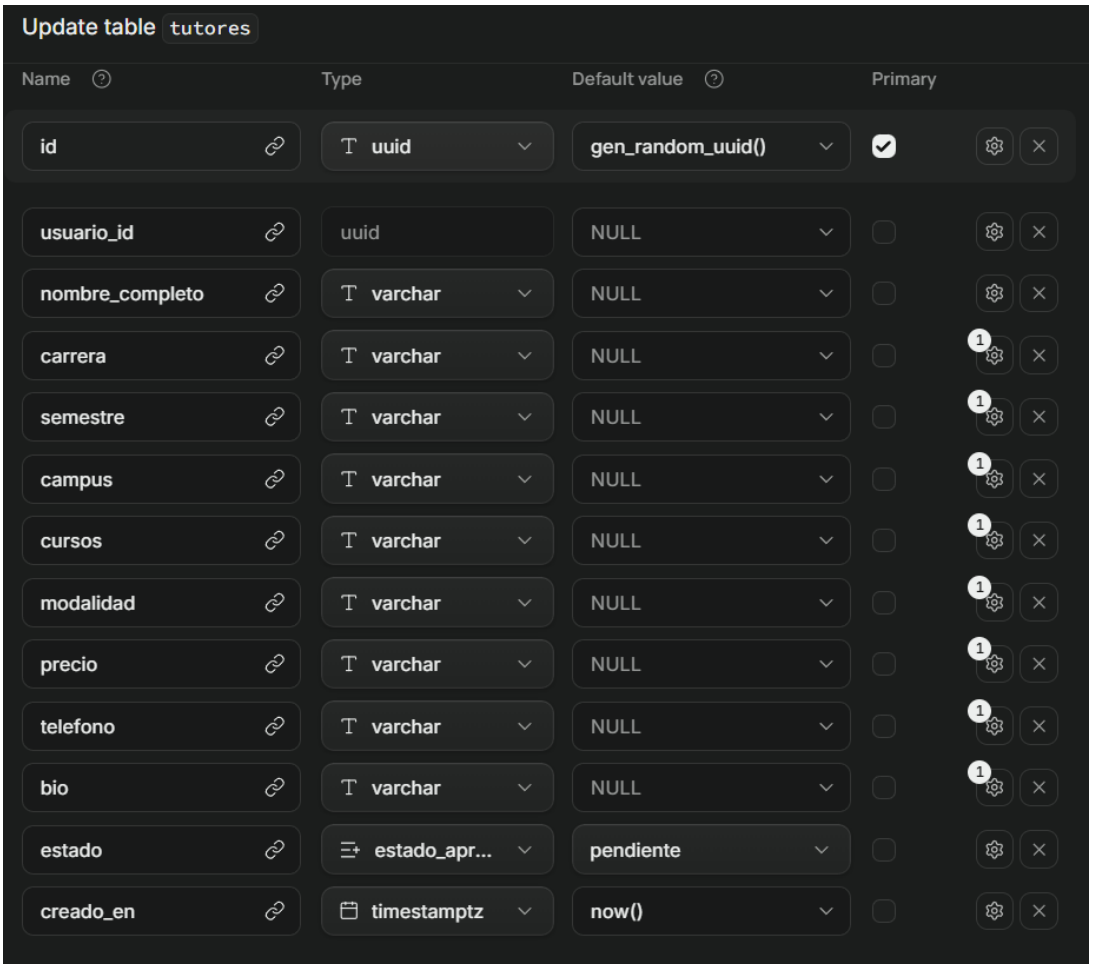


Figura 4: Estructura de la tabla tutores en Supabase.

6.3. Variables de entorno y control de versiones

Las credenciales de acceso se leen del archivo `.env`, por lo que no están expuestas en el código fuente.

Código 4: Archivo backend/.env.example

```
PORT=3000
SESSION_SECRET=cambia_esto_por_una_clave_larga_y_aleatoria

# Supabase (API y Cliente)
SUPABASE_URL=https://tu-proyecto.supabase.co
SUPABASE_SERVICE_ROLE_KEY=tu_clave_de_servicio
```

Código 5: Archivo backend/.gitignore

```
node_modules/
data/users.json
```

Explicación:

- `SUPABASE_URL` y `SUPABASE_SERVICE_ROLE_KEY`: credenciales para inicializar el cliente JS de Supabase.
- `node_modules/` en `.gitignore`: el directorio de dependencias **no se sube** al repositorio. Quien clone el proyecto lo regenera con `npm install`.
- **`.env` ya no figura en el .gitignore**: por indicación del docente, el archivo `.env` sí se envía al repositorio para que se pueda levantar el servidor y testear rápidamente. En un entorno productivo real, esto sería una vulnerabilidad.

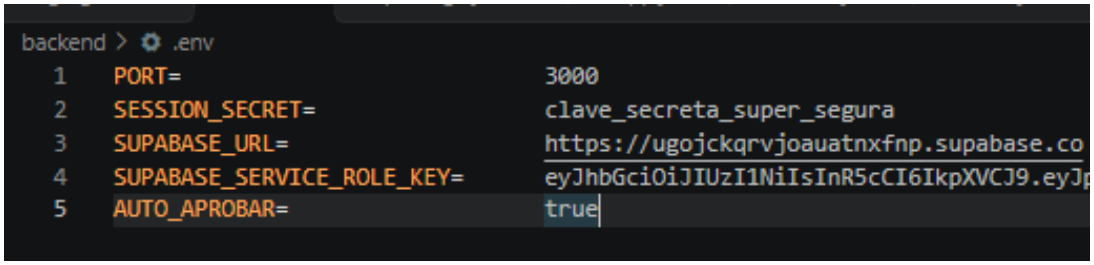


Figura 5: Archivos .env y .env.example en VS Code.

6.4. Instalación de dependencias

Código 6: Instalación del cliente de Supabase

```
cd backend
npm install @supabase/supabase-js dotenv express
```

Explicación: Descargamos el SDK oficial `@supabase/supabase-js`, lo que nos permite interactuar con la base de datos sin necesidad de escribir SQL directamente, delegando la gestión de conexiones a su API interna.

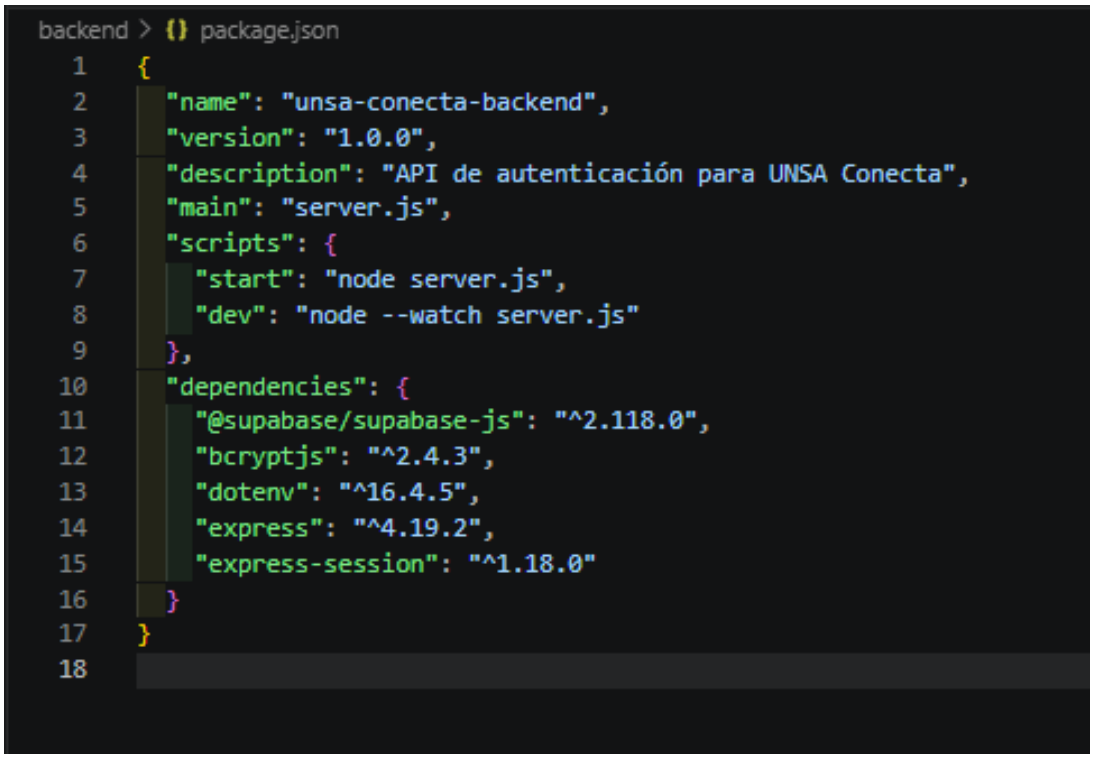


Figura 6: Dependencia supabase-js instalada.

6.5. Cliente de Supabase (`db.js`)

Código 7: backend/db.js

```
require('dotenv').config();
const { createClient } = require('@supabase/supabase-js');

const supabaseUrl = process.env.SUPABASE_URL;
const supabaseKey = process.env.SUPABASE_SERVICE_ROLE_KEY;

const supabase = createClient(supabaseUrl, supabaseKey);

module.exports = { supabase };
```

Explicación del código:

- `require('dotenv').config()`: carga las variables del `.env`.
- `createClient`: Inicializa la conexión con el proyecto de Supabase usando el Service Role Key.
- Al exportar `supabase`, todos los controladores pueden requerirlo para ejecutar los métodos del CRUD.

6.6. Controlador CRUD (`tutoresController.js`)

Todos los manejadores usan los constructores del ORM de Supabase y **ninguno** concatena variables, bloqueando intrínsecamente la Inyección SQL.

Código 8: backend/controllers/tutoresController.js - utilidades

```
const { supabase } = require('../db');

// Valida y extrae cuerpo de la peticion
function validar(body) {
    const nombre_completo = String(body?.nombre_completo || '').trim();
    const carrera = String(body?.carrera || '').trim();
    const cursos = String(body?.cursos || '').trim();
    const usuario_id = String(body?.usuario_id || '').trim(); // Foranea obligatoria

    if (!nombre_completo || !carrera || !cursos || !usuario_id) {
        return { error: 'Nombre, carrera, cursos y usuario_id son obligatorios.' };
    }
    return { datos: { nombre_completo, carrera, cursos, usuario_id } };
}

// El id debe tener formato UUID (gen_random_uuid de postgres)
const uuidValido = (id) =>
    /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i.test(id);

function manejarError(err, res) {
    console.error(err);
    res.status(500).json({ error: 'Error interno del servidor.' });
}
```

Explicación del código:

- `validar(body)`: convierte cada campo en texto y elimina espacios. Esta validación es una primera defensa a nivel de lógica de negocio.
- `uuidValido(id)`: comprueba con Regex que el parámetro de la URL sea un UUID legítimo.

Código 9: tutoresController.js - listar y obtener (SELECT)

```
exports.listar = async (req, res) => {
    try {
        const { data, error } = await supabase
            .from('tutores')
            .select('*')
            .order('estado', { ascending: true });

        if (error) throw error;
        res.status(200).json(data);
    } catch (err) { manejarError(err, res); }
};

exports.obtener = async (req, res) => {
    try {
        const { id } = req.params;
        if (!uuidValido(id)) return res.status(400).json({ error: 'ID no válido.' });

        const { data, error } = await supabase
            .from('tutores')
            .select('*')
            .eq('id', id)
            .single();
```

```
23
24          if (error) {
25              if (error.code === 'PGRST116') return res.status(404).json({ error: 'Tutor no
                    encontrado.' });
26              throw error;
27          }
28          res.status(200).json(data);
29      } catch (err) { manejarError(err, res); }
30  };
```

Explicación del código:

- `listar`: Usa `.select('*')` para traer todos los registros.
- `obtener`: Agrega `.eq('id', id)` y `.single()` para pedir exactamente un registro. Si no existe, Supabase devuelve el error PGRST116 (Not Found).

Código 10: tutoresController.js - crear (INSERT)

```
1   exports.crear = async (req, res) => {
2       try {
3           const { error: valError, datos } = validar(req.body);
4           if (valError) return res.status(400).json({ error: valError });
5
6           const { data, error } = await supabase
7               .from('tutores')
8               .insert([datos])
9               .select();
10
11          if (error) throw error;
12          res.status(201).json(data[0]);
13      } catch (err) { manejarError(err, res); }
14  };
```

Explicación del código:

- `.insert([datos])`: Inserta el objeto validado. El cliente de Supabase se asegura de escapar los valores automáticamente.
- `.select()`: Exige a Supabase devolver la fila recién insertada (con su UUID autogenerado).
- `res.status(201)`: código *201 Created*.

Código 11: tutoresController.js - actualizar y eliminar (UPDATE y DELETE)

```
1   exports.actualizar = async (req, res) => {
2       try {
3           const { id } = req.params;
4           if (!uuidValido(id)) return res.status(400).json({ error: 'ID no válido.' });
5
6           const { error: valError, datos } = validar(req.body);
7           if (valError) return res.status(400).json({ error: valError });
8
9           const { data, error } = await supabase
10              .from('tutores')
11              .update({ nombre_completo: datos.nombre_completo, carrera: datos.carrera,
                    cursos: datos.cursos })
```

```
            .eq('id', id)
            .select();

        if (error) throw error;
        if (!data || data.length === 0) return res.status(404).json({ error: 'No
            encontrado' });
        res.status(200).json(data[0]);
    } catch (err) { manejarError(err, res); }
};

exports.eliminar = async (req, res) => {
    try {
        const { id } = req.params;
        if (!uuidValido(id)) return res.status(400).json({ error: 'ID no válido.' });

        const { data, error } = await supabase
            .from('tutores')
            .delete()
            .eq('id', id)
            .select();

        if (error) throw error;
        if (!data || data.length === 0) return res.status(404).json({ error: 'No
            encontrado' });
        res.status(200).json({ ok: true });
    } catch (err) { manejarError(err, res); }
};
```

Explicación del código:

- `actualizar`: el método `.update()` modifica solo los campos enviados, filtrando con `.eq('id', id)`.
- `eliminar`: Llama a `.delete()` y devuelve la confirmación verificando que la data afectada no esté vacía mediante `.select()`.

6.7. Rutas y registro en el servidor

Código 12: backend/routes/tutores.js

```
const express = require('express');
const ctrl = require('../controllers/tutoresController');

const router = express.Router();

router.get('/tutores', ctrl.listar);
router.get('/tutores/:id', ctrl.obtener);
router.post('/tutores', ctrl.crear);
router.put('/tutores/:id', ctrl.actualizar);
router.delete('/tutores/:id', ctrl.eliminar);

module.exports = router;
```

Código 13: backend/server.js - líneas añadidas

```
const tutoresRoutes = require('./routes/tutores'); // junto a los demas require

// API
app.use('/api', authRoutes);
app.use('/api', apiRoutes);
app.use('/api', tutoresRoutes);                    // NUEVO
app.use('/api', (req, res) => res.status(404).json({ error: 'Ruta no encontrada.' }));
```

Explicación del código:

- `express.Router()`: crea un grupo de rutas independiente que mapea cada verbo HTTP hacia su respectiva función lógica.
- `:id` en la URL es un parámetro dinámico extraíble con `req.params.id`.
- `app.use('/api', tutoresRoutes)`: monta el router, originando endpoints como `/api/tutores`.

```
PS C:\Users\Admin\Desktop\uc\backend> npm start

> unsa-conecta-backend@1.0.0 start
> node server.js

UNSA Conecta en http://localhost:3000
```

Figura 7: Servidor Express en ejecución.

6.8. Pruebas de la API y Verificación en Supabase

Para comprobar la integridad de las rutas CRUD, se realizaron las peticiones y se validaron los cambios reflejados **en tiempo real en el entorno de Supabase (Table Editor)**. La Tabla 4 resume los casos y los códigos HTTP esperados.

Petición	Caso	Respuesta
`GET /api/tutores`	Lista general de tutores	200
`GET /api/tutores/<uuid>`	Obtener un tutor por ID existente	200
`POST /api/tutores`	Registro con datos válidos	201
`POST /api/tutores`	Falta un campo obligatorio	400
`PUT /api/tutores/<uuid>`	Edición con datos válidos	200
`DELETE /api/tutores/<uuid>`	Eliminación exitosa de registro	200

Tabla 4: Casos de prueba de la API.

Código 14: Cuerpo JSON de ejemplo para enviar en POST y PUT

```
{
    "nombre_completo": "Ana Torres Quispe",
    "carrera": "Ingeniería de Sistemas",
    "cursos": "Cálculo II, Programación Web",
    "usuario_id": "d0f0abce-90fc-4c91-a6e5-4209c13b355a"
}
```

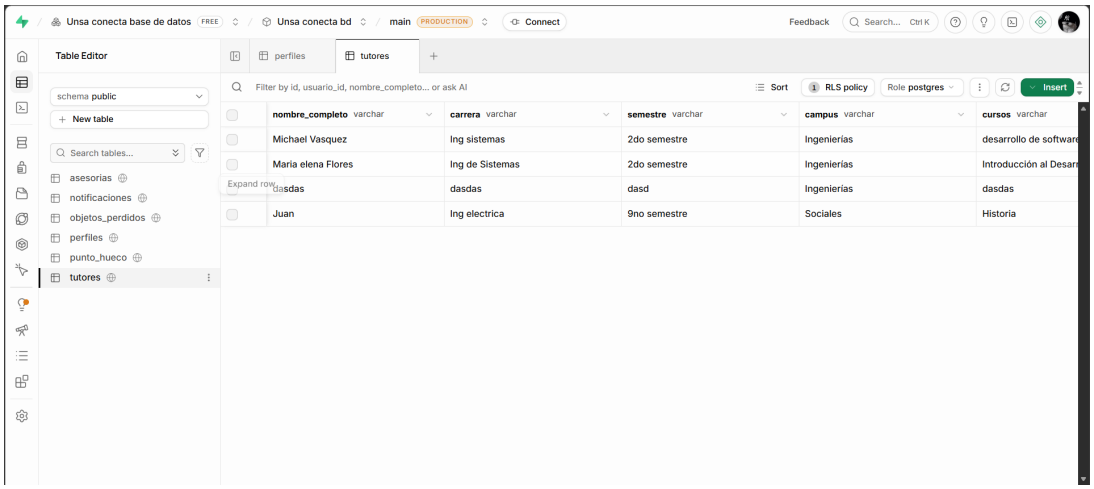


Figura 8: Verificación de lectura (GET) de Tutores.

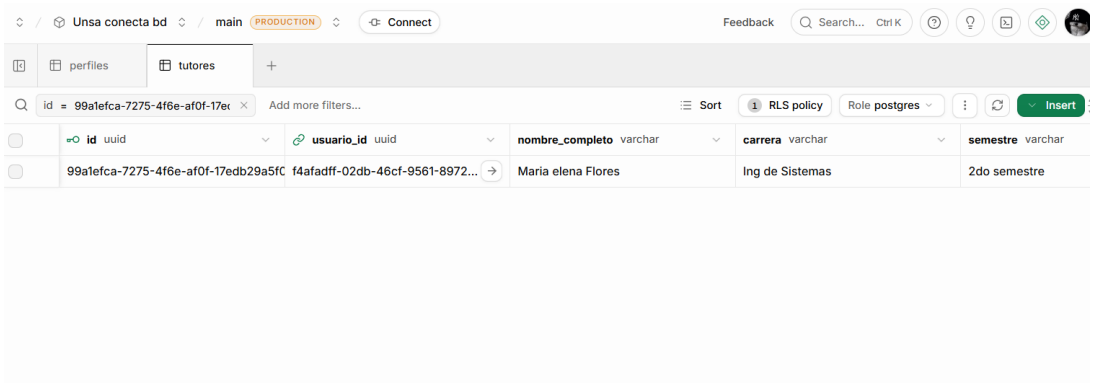


Figura 9: Prueba de lectura individual (GET /:id).

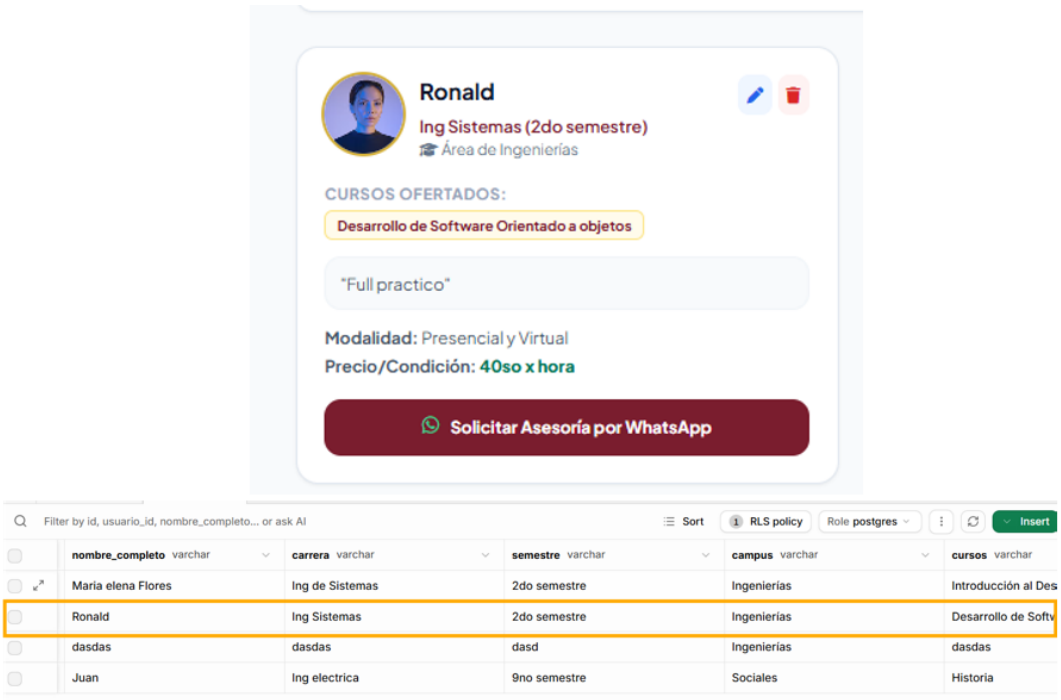


Figura 10: Verificación de inserción (POST) exitosa en Supabase.

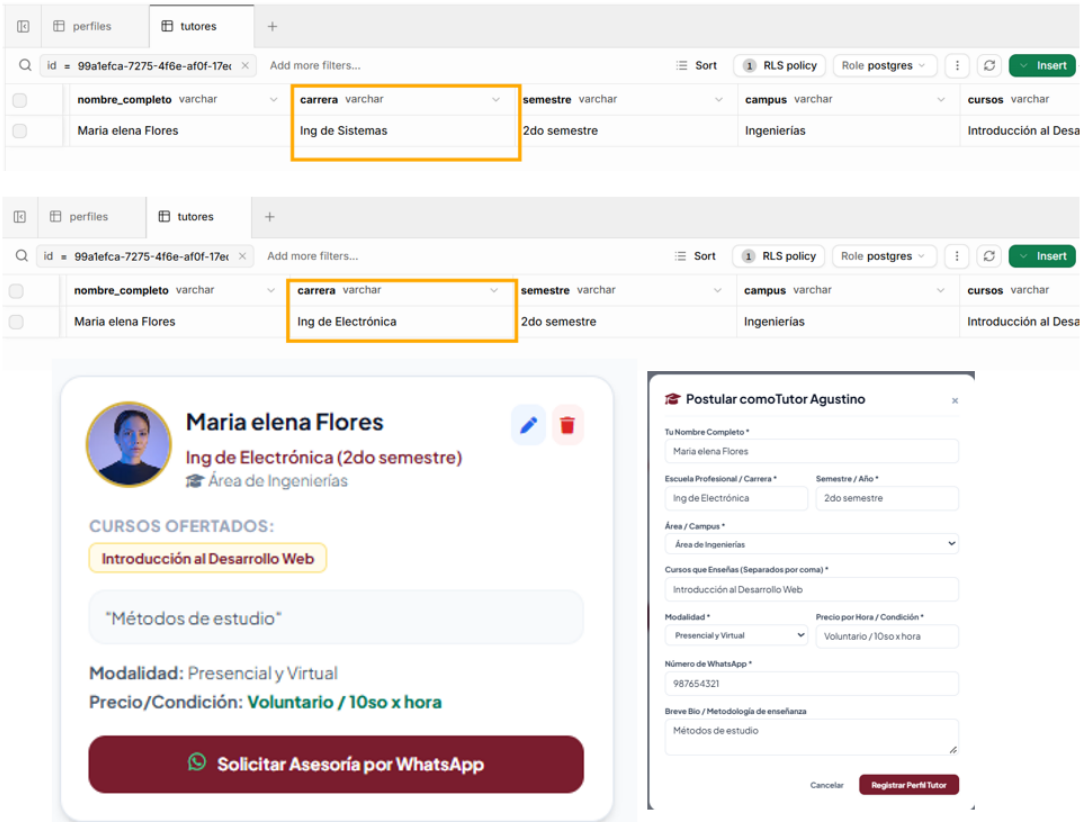


Figura 11: Verificación de edición (PUT) exitosa en Supabase.

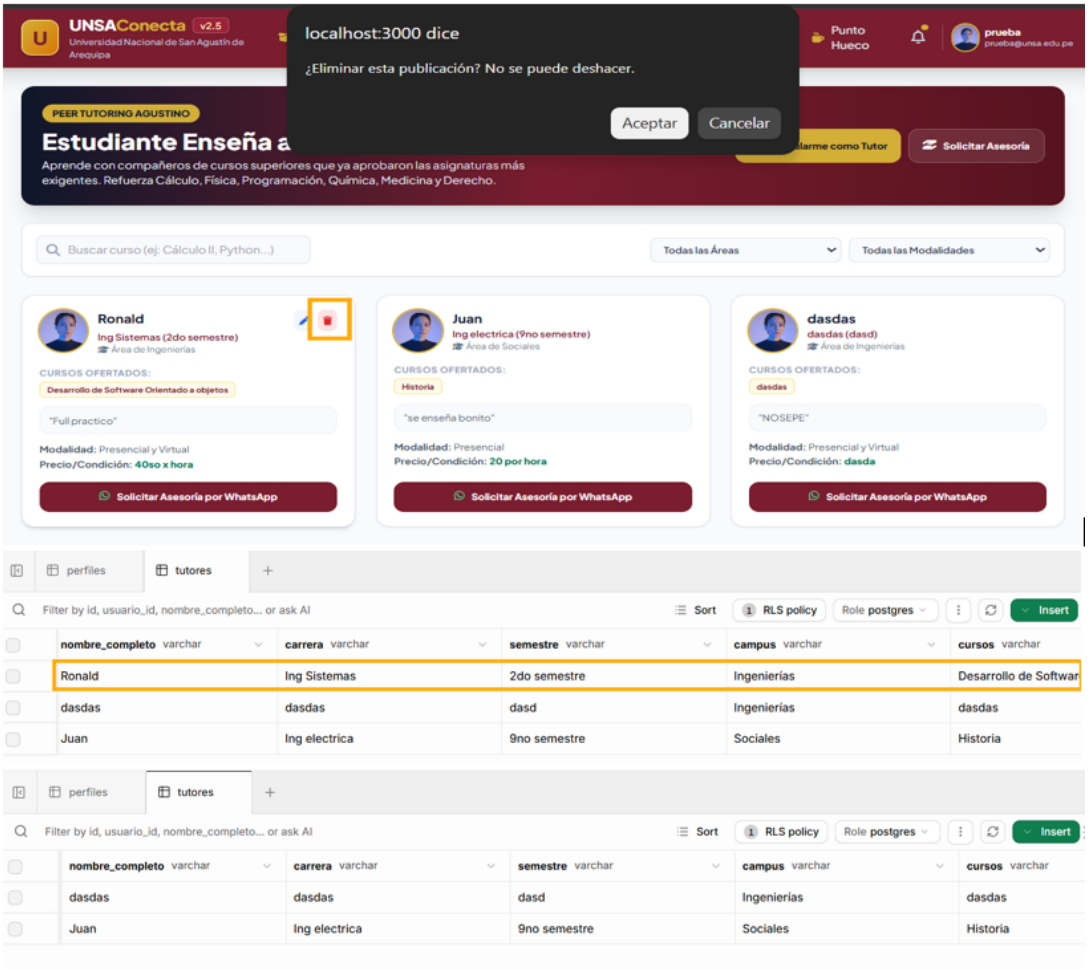


Figura 12: Verificación de eliminación (DELETE) exitosa.

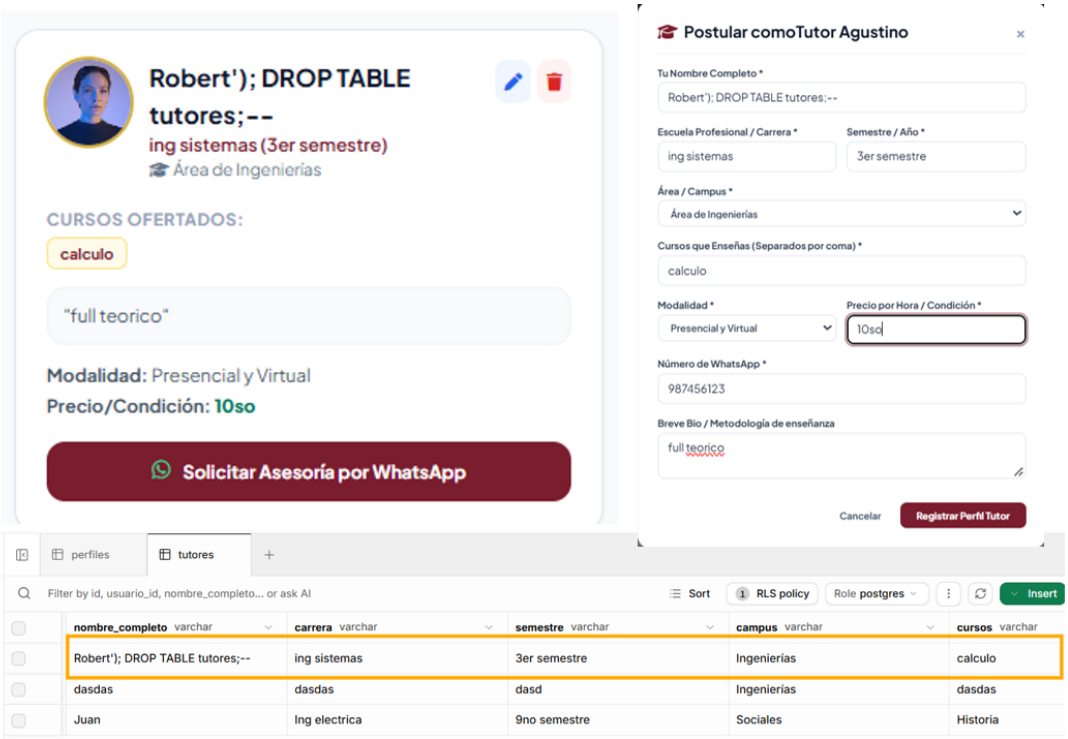


Figura 13: Validación HTTP (400 Bad Request).

6.9. Interfaz web (`tutorias.html`)

La página reutiliza el esqueleto del resto del proyecto: Tailwind CSS, el contenedor `layout-top` y `layout-bottom`, y los scripts `data.js` y `layout.js`. Se implementó el panel para mostrar y editar las tarjetas de datos de los tutores.

Código 15: fronend/tutorias.html - cuerpo de la página (resumen)

```
<body data-page="tutoring" class="h-full text-slate-800 bg-slate-50 flex flex-col
    min-h-screen">
    <div id="layout-top"></div>

    <main class="flex-grow max-w-7xl w-full mx-auto px-4 py-6 space-y-6">
        <h1 class="text-2xl font-bold">Gestión de Tutores</h1>

        <!-- Formulario de alta / edicion (Modal) -->
        <div id="tutorRegisterModal" class="hidden fixed inset-0 flex items-center
            justify-center p-4">
            <div class="bg-white rounded-xl p-6">
                <h3 id="formTitulo" class="font-bold mb-3">Registrar Tutor</h3>
                <form id="tutorRegisterForm" class="space-y-4">
                    <input id="tutNombre" type="text" required placeholder="Nombre
                        completo" class="w-full border p-2">
                    <input id="tutCarrera" type="text" required placeholder="Carrera"
                        class="w-full border p-2">
                    <input id="tutCursos" type="text" required placeholder="Cursos (ej.
                        Cálculo I)" class="w-full border p-2">
                    <div class="flex gap-2">
```

```
                        <button id="btnGuardar" type="submit" class="bg-unsa-maroon
                            text-white px-4 py-2">Guardar</button>
                        <button id="btnCancelar" type="button" class="border px-4
                            py-2">Cancelar</button>
                    </div>
                </form>
            </div>
        </div>

        <!-- Grilla dinámica -->
        <div id="tutorsGrid" class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
            <!-- se llena con JavaScript -->
        </div>
    </main>

    <div id="layout-bottom"></div>
    <script src="pub/js/data.js"></script>
    <script src="pub/js/layout.js"></script>
    <script src="pub/js/tutoring.js"></script>
</body>
```

Explicación del código:

- `data-page="tutoring"`: indica a `layout.js` qué opción del menú resaltar.
- `tutorRegisterForm`: un solo formulario sirve para **crear y editar**. El título y la lógica JS cambian su comportamiento.
- Los atributos `required` y `type` otorgan validación a nivel Frontend antes de que el usuario envíe la petición Fetch.
- `tutorsGrid`: cuerpo vacío; `tutoring.js` lo llena con los datos que devuelve la API.
- Orden de scripts: primero `data.js` (que define `apiFetch`), luego `layout.js` y al final la lógica del módulo.

6.10. Consumo de la API con Fetch (`tutoring.js`)

El proyecto ya contaba con la función `apiFetch` en `data.js`, que envuelve a `fetch` y lanza un error con el mensaje del servidor cuando la respuesta no es 2xx. Se reutilizó para no duplicar código.

Código 16: fronend/pub/js/data.js - función apiFetch (existente)

```
async function apiFetch(url, { method = 'GET', body } = {}) {
    const res = await fetch(url, {
        method,
        credentials: 'same-origin',
        headers: body ? { 'Content-Type': 'application/json' } : undefined,
        body: body ? JSON.stringify(body) : undefined
    });
    let data = null;
    try { data = await res.json(); } catch (e) { /* sin cuerpo */ }
    if (!res.ok) {
        const err = new Error((data && data.error) || 'Error inesperado (' + res.status +
            ')');
        err.status = res.status;
        throw err;
```

```
    }
    return data;
}
```

Explicación del código:

- `JSON.stringify(body)` y la cabecera `Content-Type: application/json`: serializan el objeto JavaScript para que el middleware de Express (`express.json()`) lo entienda en los POST/PUT.
- `if (!res.ok)`: `fetch` no lanza error en Catch por códigos HTTP de error lógicos (404, 500). Por eso se evalúa `res.ok` y se dispara un throw personalizado.

Código 17: fronend/pub/js/tutoring.js - carga y pintado (GET)

```
let tutores = [];    // copia local de lo que devolvio la API
let editandoId = null; // null = modo crear; UUID = modo editar

async function loadTutors() {
    try {
        tutores = await apiFetch('/api/tutores');
        renderTutors();
    } catch (err) { handleApiError(err); }
}

function renderTutors() {
    const grid = document.getElementById('tutorsGrid');
    if (tutores.length === 0) {
        grid.innerHTML = '<p class="col-span-3 text-center">No hay tutores
            registrados.</p>';
        return;
    }
    grid.innerHTML = tutores.map(t => `
        <div class="bg-white rounded-xl border p-5 shadow-sm">
            <h3 class="font-bold text-lg">${escapeHtml(t.nombre_completo)}</h3>
            <p class="text-sm text-unsa-maroon">${escapeHtml(t.carrera)}</p>
            <p class="text-xs bg-slate-100 p-2 mt-2">${escapeHtml(t.cursos)}</p>
            <div class="mt-3 flex gap-2">
                <button onclick="iniciarEdicion('${t.id}')" class="text-blue-600
                    text-sm">Editar</button>
                <button onclick="eliminarTutor('${t.id}')" class="text-red-600
                    text-sm">Eliminar</button>
            </div>
        </div>`).join('');
}
```

Explicación del código:

- `await apiFetch('/api/tutores')`: petición GET asíncrona; la ejecución espera la respuesta sin bloquear la página HTML.
- `renderTutors`: recorre el arreglo con `map` e inserta tarjetas HTML usando `innerHTML`.
- **Prevención XSS**: Se utiliza `escapeHtml()` antes de inyectar los datos en el DOM, impidiendo que scripts maliciosos insertados por el usuario afecten a otros clientes.

Código 18: tutoring.js - crear y editar (POST y PUT)

```
async function handleSaveTutor(event) {
    event.preventDefault(); // evita el submit clasico HTML
    const payload = {
        usuario_id: "fake-uuid-perfil", // (Asignado estatico para propósitos de test)
        nombre_completo: document.getElementById('tutNombre').value,
        carrera: document.getElementById('tutCarrera').value,
        cursos: document.getElementById('tutCursos').value
    };
    try {
        if (!editandoId) {
            await apiFetch('/api/tutores', { method: 'POST', body: payload });
            openNotificationToast('Tutor registrado.');
        } else {
            await apiFetch('/api/tutores/' + editandoId, { method: 'PUT', body: payload });
            openNotificationToast('Tutor actualizado.');
        }
        limpiarFormulario();
        await loadTutors(); // Refresco instantáneo del DOM
    } catch (err) { handleApiError(err); }
}

function iniciarEdicion(id) {
    const tut = tutores.find(t => t.id === id);
    if (!tut) return;
    editandoId = id;
    document.getElementById('tutNombre').value = tut.nombre_completo;
    document.getElementById('tutCarrera').value = tut.carrera;
    document.getElementById('tutCursos').value = tut.cursos;

    document.getElementById('formTitulo').textContent = 'Editar Tutor';
    openModal('tutorRegisterModal');
}
```

Explicación del código:

- `event.preventDefault()`: impide que el navegador intente un cambio de URL mediante el form submit, entregándole el control 100 % al script Fetch.
- `editandoId`: La presencia de este UUID dictamina si se ejecuta una solicitud `POST` o `PUT`.
- `await loadTutors()`: **Solo si la petición al servidor responde éxito (2xx)**, se vuelve a descargar la lista y se repinta la interfaz, reflejando siempre la realidad en PostgreSQL.

Código 19: tutoring.js - eliminar (DELETE)

```
async function eliminarTutor(id) {
    if (!confirm('¿Desea eliminar el registro?')) return;
    try {
        await apiFetch('/api/tutores/' + id, { method: 'DELETE' });
        openNotificationToast('Registro eliminado exitosamente.');
        await loadTutors();
    } catch (err) { handleApiError(err); }
}

// Inicializacion general
```

```
11 document.addEventListener('DOMContentLoaded', () => {
12     document.getElementById('tutorRegisterForm').addEventListener('submit',
           handleSaveTutor);
13     loadTutors(); // Primera carga automatica
14 });
```



Figura 14: Interfaz: panel dinámico SPA.

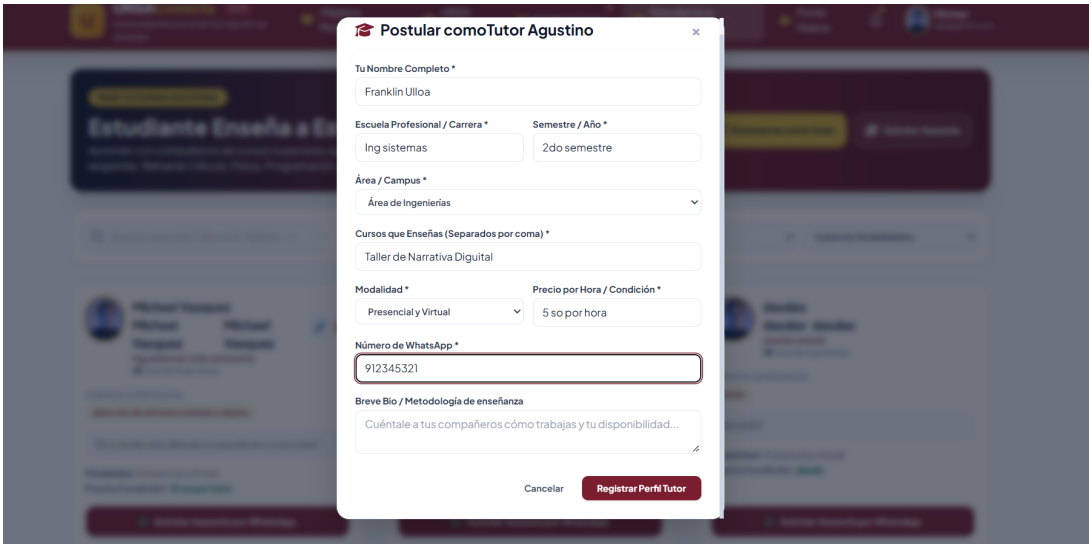


Figura 15: Interfaz: registro de tutor.

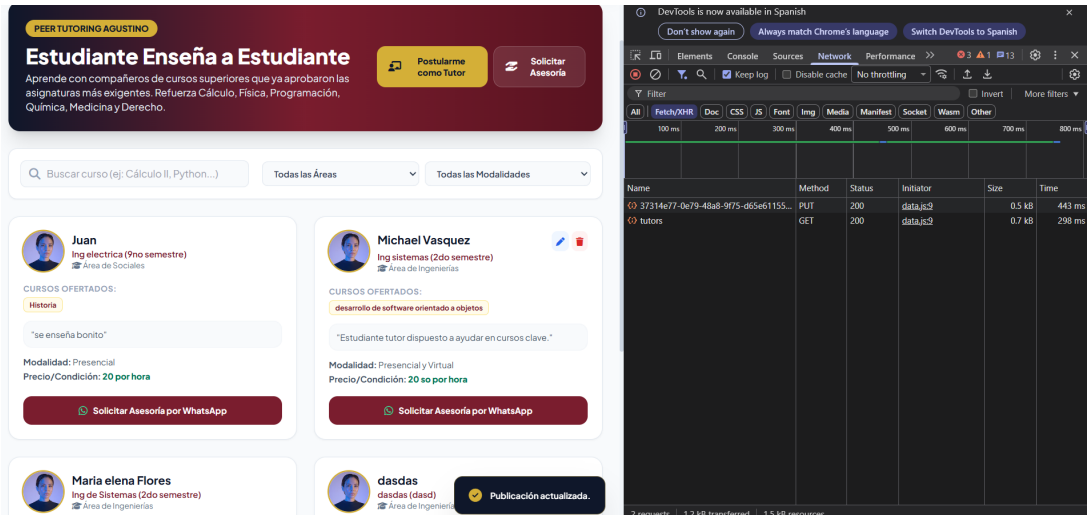


Figura 16: Interfaz: edición por PUT.

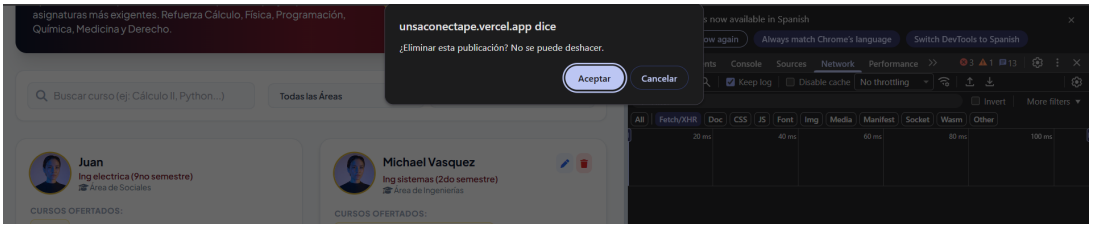


Figura 17: Interfaz: eliminación por DELETE.

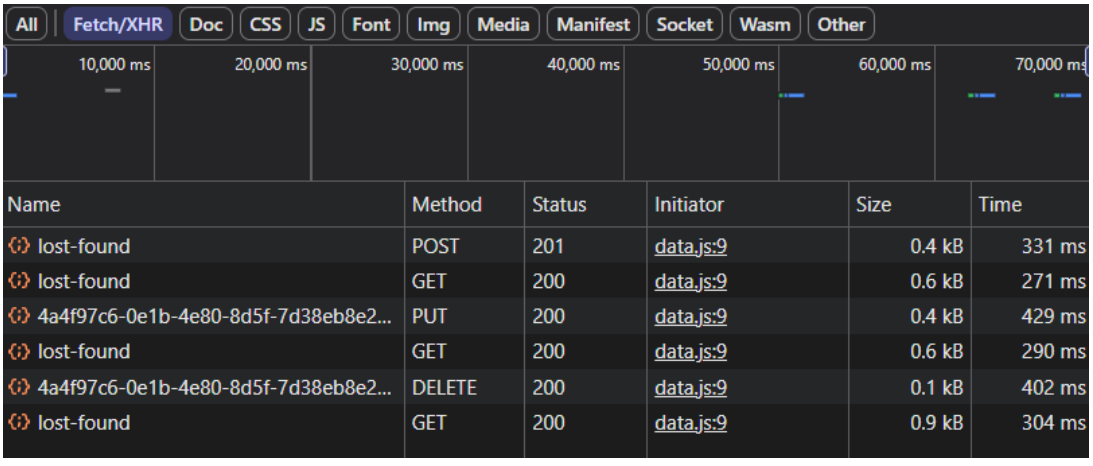


Figura 18: Peticiones Fetch en las herramientas de desarrollo.

7. Manual de Instalación, Ejecución y Uso

Esta sección permite al profesor ejecutar y probar la aplicación localmente. Antes de la entrega, el equipo clonó la rama `main` en una carpeta nueva y verificó que todo funcionara siguiendo exactamente

estos pasos.

7.1. Requisitos previos

- Node.js (versión 18 o superior) y npm.
- Acceso web al Dashboard de Supabase con permisos sobre la base de datos conectada.
- Git.

7.2. Pasos de instalación

Código 20: Instalación y ejecución desde cero

```
# 1) Clonar el repositorio (solo la rama main)
git clone https://github.com/MaicolVasquez/uc.git
cd uc

# 2) Configurar las tablas y tipos en Supabase
#    Abrir el SQL Editor del dashboard del proyecto en Supabase,
#    pegar el contenido del archivo bd/bd.sql y presionar "RUN".

# 3) Instalar dependencias del backend
cd backend
npm install

# 4) Configurar las Variables de Entorno (.env)
#    (El archivo ya viene incluido en el repositorio para evitar
#    bloqueos. Solo basta comprobar que las llaves de Supabase sean correctas)

# 5) Iniciar el servidor
npm start
```

Explicación de los pasos:

1. `git clone`: descarga el proyecto; el profesor obtiene la rama `main`.
2. Supabase: Se ejecutan los scripts DDL desde la nube. No se requiere una instalación física de base de datos en la PC del evaluador.
3. `npm install`: instala las dependencias del `package.json`, incluyendo Express y `@supabase/supabase-js`.
4. El archivo `backend/.env` **sí se incluye** intencionalmente por directiva de la materia.
5. `npm start`: inicia el servidor. La consola debe mostrar `UNSA Conecta en http://localhost:3000`.

7.3. Guía de uso de la aplicación (Frontend)

1. Abrir el navegador e ingresar a `http://localhost:3000/tutorias.html`.
2. **Listado (GET):** La grilla se auto-poblará descargando los tutores (y sus UUIDs ocultos) desde Supabase.
3. **Creación (POST):** Click en "Postularme como Tutor", rellenar los datos y pulsar Guardar. Aparecerá la nueva tarjeta sin recargar la página Web.

4. **Modificación (PUT):** Al pulsar el hipervínculo .Editar.en una tarjeta existente, la UI se readaptará para modificar. Guardar inyectará el cambio al motor de PostgreSQL de inmediato.
5. **Supresión (DELETE):** El botón .Eliminar"de una tarjeta pedirá confirmación de JavaScript, y borrará la entrada de Base de Datos y de Pantalla mediante una petición asíncrona.

7.4. Solución de problemas frecuentes

- `Auth error / Invalid Key`: Las variables `SUPABASE_URL` o la Key están mal configuradas en el `.env`.
- `relation "tutores"does not exist`: Omitió el paso 2 del manual de correr el `bd.sql` en la consola de Supabase.
- `ECONNREFUSED`: El usuario no ha iniciado el proceso `npm start` o hay conflictos en el puerto 3000 local.

8. Medidas de Seguridad Aplicadas

Medida	Cómo se implementó
Abstracción de Inyección SQL	Uso de los métodos de filtro (`.eq`) e inserción del cliente ORM de Supabase, que parametrizan la entrada automáticamente por detrás.
Gestión optimizada de conexiones	Delegada a la API PostgREST de Supabase, reemplazando la necesidad de un Pool local en el servidor de Node.
Credenciales fuera del código	Datos de conexión (`SUPABASE_URL`, Keys) leídos del `.env` mediante `process.env`.
Integridad en la base de datos	Uso de Tipos `ENUM`, llaves UUID y `ON DELETE CASCADE` aplicados desde Supabase.
Validación de entrada en Node	Campos obligatorios y validación estricta de Regex para interceptar solicitudes de Ids anómalas (filtro UUID).
Errores controlados	Códigos HTTP lógicos (400, 404, 500) con mensajes limpios; el stacktrace de error solo se registra en consola terminal.
XSS en el cliente	Uso iterativo de la utilidad `escapeHtml()` al inyectar resultados FETCH en el HTML con `innerHTML`.
Datos como JSON	Cuerpo asíncrono serializado con `JSON.stringify` e interpretación adecuada con Express Middleware.

Tabla 5: Resumen de medidas de seguridad del módulo.

Limitaciones identificadas y mejoras futuras:

- En la siguiente etapa se protegerán todas las rutas PUT/DELETE valiéndose del sistema Auth ya existente en el repositorio (validando la cookie de sesión).
- El archivo `.env` fue versionado por mandato para este Lab, sin embargo, en un entorno de CI/CD esto representaría un anti-patrón de seguridad crítico.

9. Cuestionario

1. **¿Cómo ocurre exactamente una vulnerabilidad por Inyección SQL y por qué el uso de Consultas Parametrizadas (Prepared Statements) neutraliza completamente este vector de ataque?**

   Cómo ocurre. La vulnerabilidad aparece cuando el programa construye la sentencia SQL concatenando texto que escribió el usuario. El motor de base de datos recibe un solo texto y no puede distinguir qué parte es código del programador y cuál es dato del usuario; si el usuario incluye comillas o palabras reservadas, su entrada pasa a ser código SQL.

   Por qué la parametrización (o el uso de ORMs) lo neutraliza. Al utilizar el cliente oficial de Supabase, la estructura de la consulta y los valores se procesan de forma estrictamente separada a través de su API REST interna. Los valores son pasados al motor de base de datos exclusivamente como datos, no como instrucciones ejecutables. Por lo tanto, aunque un usuario envíe palabras como `DROP TABLE`, solo serán tratadas como un texto inofensivo que se guarda en una columna.

2. **Explique la ventaja técnica de utilizar un Pool de Conexiones frente a la creación e instanciación de una nueva conexión por cada petición HTTP individual.**

   Abrir una conexión directa a una base de datos PostgreSQL requiere el establecimiento de la conexión TCP, la negociación TLS/SSL y la autenticación. Si cada petición HTTP en Node abriera y cerrara su propia conexión, la aplicación sufriría latencias enormes y colapsaría bajo carga.

   Un pool resuelve esto manteniendo un conjunto de conexiones ya abiertas y reciclables:

   - **Reutilización:** cada consulta toma una conexión libre, la usa y la devuelve, evitando repetir el "handshake"TCP.
   - **Límite de recursos:** previene que picos de tráfico generen cientos de conexiones que agoten los límites del motor de la BD.
   - **Supabase (PostgREST):** Al usar `@supabase/supabase-js`, **esta ventaja se maximiza**: no es necesario que el desarrollador gestione el Pool en Node.js; la API PostgREST de Supabase se encarga automáticamente del connection pooling (PgBouncer/Supavisor) directamente en la infraestructura de la nube, garantizando altísima concurrencia.

3. **¿Qué es la política CORS (Cross-Origin Resource Sharing) en navegadores web y cómo se configura en Express para permitir solicitudes desde dominios o puertos distintos?**

   Por seguridad, los navegadores aplican la *política del mismo origen*: un script cargado desde un origen (ej. frontend en Vercel) solo puede descargar datos de ese mismo origen. Un origen es la combinación de protocolo, dominio y puerto. **CORS** es el mecanismo que permite relajar esa restricción: el backend NodeJS indica, mediante cabeceras HTTP como `Access-Control-Allow-Origin`, qué orígenes externos y métodos (GET/PUT) están autorizados a interactuar con él.

   En Express se configura instalando y usando el paquete Middleware `cors`:

   Código 21: Configuración de CORS en Express (referencia)

```
const cors = require('cors');

app.use(cors({
    origin: 'http://localhost:5500',            // Dominio frontend permitido
    methods: ['GET', 'POST', 'PUT', 'DELETE'],
```

```
    allowedHeaders: ['Content-Type'],
    credentials: true                          // Permite leer cookies
}));
```

Notas: **En este laboratorio no fue obligatorio activar middleware CORS explícito**, porque Express sirvió ambos rubros (El HTML frontend en `express.static` y la API REST bajo `/api/`) desde un mismo entorno Node.js (`http://localhost:3000`), por lo que a los ojos del navegador web (Chrome/Firefox) no existió una violación al "Same-Origin Policy.ªl hacer un `await fetch('/api/...')` relativo.

10. Conclusiones

- Se modeló íntegramente la arquitectura de bases de datos mediante **Supabase**, garantizando la integridad de registros a través de restricciones `NOT NULL`, llaves `UUID`, ENUMs nativos y Foráneas en Cascada.
- Se conectó la capa intermedia NodeJS/Express usando el cliente oficial `@supabase/supabase-js`, lo que abstrae la complejidad de mantener Pools locales de conexiones y centraliza el manejo de datos en su API REST segura (PostgREST).
- Los endpoints CRUD demostraron resistencia absoluta ante inyecciones SQL gracias a la parametrización intrínseca de los métodos del ORM de Supabase (`.insert`, `.eq`), forzando la interpretación literal de strings anómalos.
- La interfaz web de "Tutoríasïnteractúa activamente con el servidor valiédnose de funciones HTTP asíncronas (`fetch()`). De la mano de una política de manipulación prudente del DOM en el callback local, se obtuvieron operaciones sin recargas de vista completas.
- El trabajo metodológico en ramas, la omisión consciente del `node_modules`, la inclusión pedagógica del `.env` y la redacción del manual de ejecución, preparan el camino para que el evaluador reproduzca la arquitectura Full-Stack sin inconvenientes.

11. Rúbrica

11.1. Criterios de evaluación

Tabla 6: Rúbrica de evaluación del Laboratorio 08.

Criterios		Puntos	Check-list	Estudiantes	Profesor
Persistencia (35 %)	Conecta Node.js a BD con Cliente Oficial. Implementa CRUD mitigando SQLi con métodos del ORM.	7	X	7	
Integración FrontEnd con Fetch API (35 %)	Consume la API REST con async/await y Fetch API. Actualiza el DOM cliente sin refresco de página ante cada operación CRUD.	7	X	7	
Modelo de Base de Datos y Restricciones (15 %)	Diseño relacional sólido, tipos de datos óptimos, claves primarias y restricciones NOT NULL/UNIQUE correctamente especificadas.	3	X	3	
Cuestionario Teórico (15 %)	Responde acertadamente sobre Inyección SQL, mecanismo del Pool de conexiones y políticas CORS.	3	X	3	
Total		**20**		**20**	

12. Referencias y Bibliografía Recomendada

- PostgreSQL Global Development Group. (2025). *PostgreSQL 16 Documentation*. Postgresql.org.
- node-postgres Community. (2024). *node-postgres (pg) Official Documentation*. Node-postgres.com.
- MDN Web Docs. (2025). *Using the Fetch API & CORS*. Developer Mozilla.
- OWASP Foundation. *SQL Injection Prevention Cheat Sheet*. OWASP Cheat Sheet Series. OWASP.org.
- Express.js Community. *Express: Fast, unopinionated, minimalist web framework for Node.js*. Documentación oficial.